Assignment1

好的，这是 **assignment1.md** 文档的完整中文翻译，包括所有示例。

Assignment 1

要求

本次作业要求学生设计并实现（或改进）一种使用AI方法（例如大语言模型）的测试技术。学生可以选择静态测试、黑盒动态测试或白盒动态测试技术。

注意：本页列出了静态代码分析工具列表；黑盒测试技术包括等价类划分、边界值分析、输入组合测试、状态转换测试模型生成器、决策表测试；白盒测试可以测量语句覆盖、分支覆盖、条件覆盖、路径覆盖、定义-使用覆盖等。

现在，作业要求学生将其实现为一个工具，该工具可以接受两种形式的输入：(1) 系统需求；(2) 测试代码库。该工具应能分析 (1) 系统需求 或 (2) 测试代码库（或部分模块）以创建测试用例。

提交制品

1. **输入**：需求 / 项目代码库
2. **工具制品**：使用的提示词、使用的模型、模型生成的代码
3. **生成的输出**：报告的告警（针对静态分析）；测试用例（针对黑盒和白盒分析）
4. **实验分析**（准确性、覆盖率、泛化性等）
5. **项目报告**
    a. 与传统非AI技术的比较，优缺点
    b. 分析报告：在实践中如何遇到AI的局限性以及如何改进工具？
    c. 总结

评估标准

- 概念理解 10%
- 设计与实现的一致性 20%
- 覆盖率与有效性/实用性 40%

- 深入分析（泛化性演示、推理等） 20%
- 汇报展示 10%

汇报展示

每个小组有15分钟时间完成项目英文汇报，涵盖上述所有方面。随后是问答环节。评审将根据提交的文档、汇报内容以及学生应掌握的软件测试基础知识进行提问。

小组成员的贡献分配默认均等。任何偏差必须单独申请，并获得所有成员签名批准。

提交

每个小组必须在汇报日期前一天通过电子邮件向助教提交以下材料：

1. **提交制品**：包含上述所有要求内容（封面页必须包含团队ID、所有成员的全名和学号）。
2. **最终汇报PPT**：第一张幻灯片必须包含团队ID以及所有成员的全名和学号。

报告和PPT必须以 **PDF格式** 提交，测试脚本必须作为压缩文件提交。

提交与汇报日程：

- **提交截止日期**：第8周，星期一，下午17:00前
- **汇报日期**：第8-9周，星期二/星期四，10:00–11:35

示例提交 Ex1：

标题：基于LLM的多商品智能售货机动态黑盒测试

1. **输入：**
    - **系统概述**：被测系统是一台部署在公共场所（如地铁站）的智能售货机。其内部软件、硬件控制逻辑和数据库对测试者不可见。测试仅基于外部可观察的行为和提供的需求进行。
    - **功能需求**：
        - **商品选择**。售货机提供三类产品：饮料（价格范围： 1.50, 3.00）；零食（价格范围： 2.00, 4.50），热食（价格范围： 5.00, 10.00）。
        - **接受的支付方式**。售货机接受多种支付方式：1. 硬币： 0.10, 0.25, 0.50, 1.00；2. 纸币： 5.00, 10.00。

- **支付约束**。总投入金额必须大于或等于商品价格。机器最多只能找零 5.00；拒绝需要找零超过 5.00的交易。
- **库存约束**。商品可能在支付过程中售罄。

2. **工具制品：**
   - **使用的LLM**：GPT-4o
   - **使用的提示词：**

> 你是一个软件测试助手。给定以下系统概述和需求，请识别：
>
> 输入变量 2) 等价类划分（有效和无效） 3) 边界值 4) 具体测试用例
> 需求：{requirement_text}

1. **生成的输出**
   - **等价类划分：**

ID	描述	结果
EP1	投入金额 < 商品价格	有效
EP2	投入金额 > 商品价格	无效
…		

   - **边界值分析：**

边界	值
商品价格	1.4, 1.5, $1.6
支付总额	0, 1.4, $15.1
…	

   - **示例测试用例：**

测试用例	场景	预期结果
TC1	零食 3.00，支付 8.50	拒绝（找零 > $5）
TC2	零食 3.00，支付 3.50	支付成功，找零 $0.50
…		

2. **实验分析**
   - 4.1 EP/BVA和测试用例的覆盖率。
   - 4.2 缺失项分析。
   - 4.3 为改进覆盖率、准确性和泛化性而优化提示词。
3. **项目报告**
   - 与传统非AI技术的比较，优缺点
   - 分析报告：AI的局限性以及改进工具的方法
   - 总结

示例提交 Ex2：

标题：基于LLM的静态分析

1. **输入：**
   - **系统概述**：
     Axios是一个基于Promise的网络请求库，可以在Node.js和浏览器中运行。该库改编自原始Axios v1.3.4版本，使其兼容OpenHarmony，同时保留其现有的使用模式和功能。
     - HTTP请求
     - Promise API
     - 请求和响应拦截器
     - 请求和响应数据的转换
     - 自动转换JSON数据
       Axios的源代码可以从
       https://ohpm.openharmony.cn/#/cn/detail/@ohos%2Faxios 获取。
2. **工具制品**

- **示例提示词：**

> 你是一个静态代码分析器。分析以下 {language} 代码并检测潜在问题。
> 识别以下内容：
>
> 语法错误
>
> 安全漏洞
>
> 已弃用或不兼容的API使用
>
> 潜在的运行时错误
>
> 代码质量问题
> 以结构化的JSON格式返回结果：
> {
> "issues": [
> {
> "line": ,
> "type": "",
> "description": "<详细描述>",
> "severity": ""
> }
>]
> }
> 代码：
> {source_code}

1. **生成的输出**

- **问题 1：**

Plain Text

```
[{
    "line": 4,
    "type": "资源管理",
    "description": "使用'open'打开文件但未使用上下文管理器管理。如果发生异常，文件可能保持打开状态。",
    "severity": "中等",
    "recommendation": "使用 'with open(filename, 'r') as f:' 来自动关闭文件。",
    "category": "代码质量",
    "reference": "https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files "
```

```
}]
```

- **测试用例（概念验证）：**

2. **实验分析**
   - 4.1 误报分析
     - 优化提示词以提高准确性
     - 尝试更多项目以提高泛化性
     - 开发者的Bug报告与验证
3. **项目报告**
   - 与传统非AI技术的比较，优缺点
   - 分析报告：AI的局限性以及改进工具的方法
   - 总结

示例提交 Ex3：

标题：基于LLM的白盒测试

1. **输入**
   - **系统概述：**
     Axios是一个基于Promise的网络请求库，可以在Node.js和浏览器中运行。该库改编自原始Axios v1.3.4版本，使其兼容OpenHarmony，同时保留其现有的使用模式和功能。
     - HTTP请求
     - Promise API
     - 请求和响应拦截器
     - 请求和响应数据的转换
     - 自动转换JSON数据
       Axios的源代码可以从
       https://ohpm.openharmony.cn/#/cn/detail/@ohos%2Faxios 获取。
2. **工具制品**
   - **提示词：**

> 你是一位专家软件测试员和白盒测试助手。
> *你的任务是分析以下 {language} 函数/模块，并生成一组能实现完全语句覆盖的测试用例。*
> 请执行以下操作：
>
> 识别代码中的所有可执行语句。
>
> 为每个语句生成至少能执行一次的测试输入值。
>
> *以适合自动化测试的结构化JSON格式输出测试用例。*
> 结构化JSON格式：
> {
> "function": "",
> "test_cases": [
> {
> "input": { "": <值>, ... },
> "expected_output": "<预期输出或行为>",
> "covered_statements": [],
> "notes": "<可选解释>"
> }
>]
> }
> 待分析的代码：
> {code_snippet}
> 注意：
>
> 如果需要覆盖条件语句的所有分支，请包含边界情况。
>
> 如果某个语句由于逻辑错误而无法到达，请将其标记为“unreachable”。
>
> 提供每个测试用例如何实现语句覆盖的解释。

1. **生成的输出**
   - **被测函数：**
   - **测试用例：**
     - TestCase1: 删除指定文件
     - TestCase2: 异常处理
2. **实验分析**
   - 4.1 误报分析
     - 优化提示词以提高准确性

        - 尝试更多项目以提高泛化性
        - 开发者的Bug报告与验证

3. **项目报告**
    - 与传统非AI技术的比较，优缺点
    - 分析报告：AI的局限性以及改进工具的方法
    - 总结